

Projekt: Software Engineering

Abstract

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Making-of. MietManager ist eine PyQt6-Desktop-Anwendung, mit der private Kleinvermieter ihre Immobilien, Mieter und Mietverträge verwalten und daraus automatisiert eine versandfertige Nebenkostenabrechnung als PDF-Geschäftsbrief erzeugen. Der Zeitplan der Projektdokumentation sah für das Gesamtprojekt vier Kalenderwochen vom 27.07.2026 bis zum 23.08.2026 vor. Tatsächlich zog sich die Fertigstellung bis zum 26.09.2026, weil die Bearbeitung neben einem Vollzeit-Studium und einer Berufstätigkeit nur abends und am Wochenende möglich war. Der Kern der Implementierung fand dagegen konzentriert innerhalb einer Woche statt, wie sich anhand der Commit-Historie nachvollziehen lässt.

Vor der eigentlichen Implementierung stand die Konzeptionsphase mit Anforderungs- und Spezifikationsdokument. Für die Benutzerschnittstellen entstanden dabei zunächst handskizzierte und daraus abgeleitete digitale GUI-Mockups der zentralen Ansichten, darunter das Dashboard, die tabellarischen Datenansichten für Immobilien, Mieter, Mietverträge und Kostenpositionen, der Nebenkostenabrechnung-Tab und das Profil-Popup. Diese Mockups legten Struktur und Aufbau der wichtigsten Dialoge bereits vor dem ersten Code fest und dienten während der Implementierung als Vorlage für die tatsächliche Oberfläche.

Nach dieser Planungsphase bildete das Grundgerüst mit Projekt-Setup, das SQLAlchemy-Datenmodell für die acht Geschäftsobjekte sowie Datenbank-Engine, Session-Handling und ein Seed für Testdaten. Anschließend folgten die Geschäftsregeln und die Berechnungslogik der Nebenkostenabrechnung, bewusst vor der Oberfläche und direkt mit ersten automatisierten pytest-Tests abgesichert, sodass die Service-Schicht von Beginn an unabhängig von Qt testbar war. Am 14.08.2026 entstand die grafische Oberfläche, wobei zunächst Hauptfenster, Navigation und Theming und danach die CRUD-Tabs und -Dialoge für Immobilien, Mieteinheiten, Mieter und Mietverträge und anschließend der Nebenkostenabrechnung-Tab mit Vermieterprofil erstellt wurden. Anschließend wurde die Anwendung gegen Fehler gehärtet. Sie wurde als eigenständige Windows-.exe mit PyInstaller gebaut, die automatisierten Tests wurden auf rund 100 % Abdeckung der Service-Schicht ausgeweitet, ein Absturz beim Löschen eines bereits abgerechneten Mietvertrags wurde behoben und mehrere UX-Schwächen wurden ausgeräumt, wie in der anschließenden Reflexion erläutert.

Reflexion. Die frühe Entscheidung für eine strikte Schichtentrennung zahlte sich aus. Als sich zeigte, dass Geschäftsregeln zunächst nur innerhalb der UI-Schicht geprüft wurden, ließ sich die Logik ohne Umbau der Oberfläche in das bereits bestehende, von Qt und Datenbank unabhängige Service-Modul auslagern und dort automatisiert testen.

Mehrere Probleme zeigten sich erst während der Nutzung der eigenen, bereits laufenden Anwendung. So traten wiederholt visuelle Fehler in der Benutzeroberfläche auf. Deutlich wurde außerdem, dass die anfangs in jedem Tab vorhandenen manuellen Aktualisieren-Schaltflächen die Bedienung unnötig

erschweren. Nutzer mussten nach jeder Änderung selbst aktiv werden, damit sie an anderer Stelle sichtbar wurde. Als Reaktion darauf wurden diese Schaltflächen in allen Tabs entfernt und durch eine automatische Aktualisierung beim Tabwechsel ersetzt, ergänzt um eine optische Kennzeichnung nicht veränderbarer Felder durch Grautöne. Eine Anpassung, die im ursprünglichen Anforderungs- und Spezifikationsdokument nicht vorgesehen war, sondern sich erst aus der Erfahrung mit der eigenen fertigen Anwendung ergab.

Der ursprünglich geplante Zeitplan wurde nicht eingehalten. Statt der vorgesehenen vier Wochen benötigte das Projekt bis zum 26.09.2026 mehr als das Doppelte an Kalenderzeit. Ausschlaggebend war vor allem, dass die Bearbeitung neben Studium und Beruf nur in begrenzten Zeitfenstern abends und am Wochenende stattfinden konnte. Diese engen Zeitfenster verstärkten auch die im Risikomanagement der Projektdokumentation als R2 identifizierte Wartezeit auf Tutor-Feedback. Traf eine Rückmeldung etwa an einem Wochentag ein, ließ sie sich häufig erst im nächsten freien Abend- oder Wochenendfenster bearbeiten. So summierten sich die begrenzte eigene Zeit und die Wartezeiten zu einer deutlich längeren Kalenderlaufzeit, als es die anhand der Commit-Historie sichtbare, konzentrierte Implementierungswoche vermuten ließe.

Die wichtigste Erkenntnis betrifft daher die Zeitplanung selbst. Ein Zeitplan, der im Wesentlichen die eigene Bearbeitungszeit kalkuliert, unterschätzt systematisch den tatsächlichen Kalenderaufwand, wenn echte Arbeitszeit nur in schmalen Zeitfenstern neben Studium und Beruf zur Verfügung steht. Künftig sollte ein Zeitplan diese begrenzte Verfügbarkeit von Anfang an realistisch abbilden, statt von durchgehend nutzbaren Arbeitstagen auszugehen. Zweitens hat sich bestätigt, dass Testbarkeit von Beginn an in der Architektur mitgedacht werden sollte, statt sie nachträglich in eine bereits bestehende UI-Schicht hineinzuziehen. Die Implementierungsreihenfolge dieses Projekts, zuerst Geschäftslogik und Tests und erst danach die Oberfläche, hat das bereits weitgehend erfüllt. Drittens zeigte sich, dass Usability-Schwächen wie die anfänglichen manuellen Aktualisieren-Schaltflächen oft erst durch tatsächliche Nutzung der eigenen Anwendung sichtbar werden, weshalb eigenes Testen der fertigen Oberfläche fester Bestandteil des Entwicklungsprozesses sein sollte, statt sich allein auf die vorab entworfenen Mock-ups zu verlassen.

Für eine Weiterentwicklung über dieses Portfolio hinaus böten sich mehrere Erweiterungen an. Kurzfristig ließen sich die bereits als Kann eingestufteten Funktionen automatischer E-Mail-Versand und CSV-Export der Dashboard-Kennzahlen ohne Änderung der bestehenden Architektur ergänzen. Bei wachsendem Kundenstamm über die dokumentierte Skalierungsgrenze hinaus oder bei Bedarf an Mehrbenutzerbetrieb müsste die dateibasierte SQLite-Anbindung durch eine Client-Server-Datenbank ersetzt werden, wofür die bestehende Trennung von Service- und Datenzugriffsschicht bereits eine geeignete Grundlage bietet.